Correction TD
Notation en virgule flottante
Mathématiques pour l'informatique
—IMAC 2—

▸ **Exercice 1.** ***Calculs faciles***

1. *Récupérer le programme* `calculsFaciles.cpp` *ou bien recopier les lignes suivantes sur votre éditeur puis compiler. Que constatez-vous ? Quelle est votre explication ?*

```
#include <iostream>
#include <iomanip>

int main()
{
  std::cout << std::setprecision(-1) << "1.0  + 2.0  = " << 1.0 + 2.0 << std::endl;
  std::cout << std::setprecision(20) << "1.0f + 2.0f = " << 1.0f + 2.0f << std::endl;
  std::cout << std::setprecision(20) << "1.0  + 2.0  = " << 1.0  + 2.0  << std::endl;
  std::cout << std::setprecision(20) << "1.0L + 2.0L = " << 1.0L + 2.0L << std::endl;

  std::cout << std::setprecision(-1) << "0.1  + 0.2  = " << 0.1 + 0.2 << std::endl;
  std::cout << std::setprecision(20) << "0.1f + 0.2f = " << 0.1f + 0.2f << std::endl;
  std::cout << std::setprecision(20) << "0.1  + 0.2  = " << 0.1  + 0.2  << std::endl;
  std::cout << std::setprecision(20) << "0.1L + 0.2L = " << 0.1L + 0.2L << std::endl;

  std::cout << std::endl;
  std::cout << std::setprecision(20) << "0.1f - 0.1L = " << 0.1f - 0.1L << std::endl;

  return 0;
}
```

2. *Téléchargez puis compilez le programme* `seeFloat.cpp` *sur la page de l'enseignant. Testez le avec les nombres 1,2,3 et 0.1, 0.2, 0.3 (et plein d'autres tests). Qu'observez-vous?*

3. *D'après vous, quel est le résultat de l'opération suivante ?*

```
if( (0.1 + (0.2 + 0.3) == (0.1 + 0.2) + 0.3) ) std::cout << "true" << std::endl;
else std::cout << "false" << std::endl;
```

Faites un test pour vérifier.

✂ ----------

La machine n'est pas capable de représenter exactement 0.1 (et par conséquent 0.2 qui est le double, c.-à-d. pareil avec un décalage de bit dans l'exposant). 0.1 n'est ni une puissance de 2, ni une somme de puissances de 2.

---------- ✂

▸ **Exercice 2.** ***Et en C/C++ ?***

1. *Qu'est-ce que je risque en écrivant dans mon programme les lignes suivantes :*

```
float a = ... ;
float b = ... ;

if(a == b) ... ;
```

   Qu'est-ce que je devrais écrire à la place ?

2. *Suis-je en danger si j'écris :*

```
int a = ... ;
float b = a/3;
```

3. *Mon programme compile-t-il avec la ligne suivante :*

```
float a = -5.0/0;
std::cout << " a " << a << std::endl;
```

4. *Est-ce que je peux mourir si ma vie dépend de la fiabilité de ce calcul* [(1)] *:*

```
float a = powf(2.0,40);
std::cout << " a = " << a << std::endl;

int b = a;
std::cout << " b = " << b << std::endl;
```

5. *Je vais manger dès que la boucle est finie, qu'est-ce qu'on mange ?*

```
for(float a=0.0; a<10; a+=0.00000001)
  ... ;

std::cout << " à table ! "  << std::endl;
```

[(1)] question de J. Chaussard

✂ ----------

1. avec les erreurs numériques, a et b riquent d'être très proches mais différents.
2. a/3 est une division entière. Si a n'est pas un multiple de 3, b n'aura pas la valeur espérée.
3. oui ça compile, mais avec un warning, le résultat est -inf
4. le max pour un int est $2^{31} - 1$ alors que pour un float, c'est 2^{127}. Donc si on file à l'int 2^{40}, il met n'importe quoi, par exemple, un nb négatif :

   ```
   a = 1.09951e+12
   b = -2147483648
   ```

5. on ne mange rien car c'est une boucle infinie car une fois que a est grand, l'opération a += 0.00000001 donne a comme résultat.. (a bloque à 0.25 au lieu de finir à 10). Par contre, avec un double, ça marche.

---------- ✂

▸ **Exercice 3.** ***Suites***

Soit la suite définie par :

$$\begin{cases} u_0 &= \dfrac{1}{3} \\ u_{n+1} &= 4u_n - 1 \end{cases}$$

1. *calculer manuellement les 5 premiers termes.*
2. *faites un programme C++ calculant les 100 premiers termes avec des* `float` *et les 550 premiers termes avec des* `double`*.*

✂ ----------

1. 1/3, 1/3, 1/3, 1/3, ... en théorie, ça ne change jamais
2. En pratique, accumulation d'erreurs.

```
#include <iostream>
#include <iomanip>
// g++ -Wall -O2 series.cpp

template<typename T>
T next(const unsigned int & n){
if(n == 0) return  T(1.0)/T(3.0);
return T(4.0) * next<T>(n-1) - T(1.0);
}

int main(){
// float
for(int i=0; i<100; ++i)
```

```
    std::cout << std::setprecision(30) << i << " : " << next<float>(i) << std::endl;

    // double
    for(int i=0; i<550; ++i)
    std::cout << std::setprecision(30) << i << " : " << next<double>(i) << std::endl;

    return 0;
    }
```

▸ **Exercice 4.** ***Multiplication à la russe***

La multiplication à la russe $a \times b$ consiste à répéter l'opération "diviser a par 2 et multiplier b par 2 jusqu'à ce que $a = 1$", dans quel cas $a \times b = b$. Deux cas sont à considérer dans ce processus itératif : si a est pair, tout se passe bien, si a est impair, sa division par 2 génère un résidu qu'il faudra additionner au résultat final. Cette approche était utilisée sur les premières calculatrices.

Par exemple 13×320 *:*

opération	a	b	*résidu*
	13	*320*	
13/2=6 reste 1	*6*	*640*	*320*
6/2=3 reste 0	*3*	*1280*	-
3/2=1 reste 1	*1*	*2560*	*1280*

résultat : $13 \times 320 = 2560 + (320 + 1280) = 4160$

Il apparait clairement qu'il est préférable de choisir pour a la plus petite des deux valeurs à multiplier.

Coder la multiplication à la russe en C++.

Faire une explication graphique

```
int plop(int a, int b)
{
    // on gère le signe
    int signe = 1;
    if(a<0){
        signe = -signe;
        a = -a;
    }
    if(b<0){
        signe = -signe;
```

```
        b = -b;
    }

// swap
if(a>b) std::swap(a,b);

// multiplication par 0
if(a == 0) return 0;

    // calcul
    int residu = 0;
    while(a>1){
        if(a&(int)1) residu += b;
        a = a >> 1;
        b = b << 1;
    }

    return signe>0 ? (b+residu) : -(b+residu); // pas de 'signe*...' car pas d'opérateur '*'
}
```

▸ **Exercice 5.** ***Racine carrée***

1. *Imaginez un programme estimant la racine carrée d'un nombre à virgule flotante.*
2. *Newton a proposé une méthode pour estimer la racine carrée d'un nombre x en iterant sur la suite suivante :*

$$\begin{cases} u_0 = 1 \\ u_{n+1} = \dfrac{u_n + x/u_n}{2} \end{cases}$$

   Testez cette méthode et commentez.

3. *Allez voir sur wikipedia.*

---

1. Trouver $x = \sqrt{a}$
   (a) méthode brute force : trouver x tel que $x^2 = a$ en prenant $x = a/2$ au départ, puis en tatonant.
   (b) autre brute force : deep learning
   (c) autre brute force : on fait une texture ou un tableau de n racines carrées, ensuite, on interpole linérairement (ou pas) pour les valeurs intermédiaires
   (d) annuler la fonction $f(b) = a - b^2$ avec une dichotomie (méthode précedente) ou avec des méthodes plus performantes comme Newton.
   (e) tricherie : $x = \sqrt{a} \Leftrightarrow x = e^{\log(a^{\frac{1}{2}})} = e^{\frac{1}{2}\log(a)}$

(f) méthode du soroban : on fait la somme des nombres impaires jusqu'à dépasser a, on compte le nombre d'itérations, c'est une bonne approximation de x.

$$a = \sum_{i=1}^{x} 2i - 1 = 2\sum_{i=1}^{x} i - \sum_{i=1}^{x} 1 = 2\frac{x(x+1)}{2} - x = x(x+1-1) = x^2 \quad \Rightarrow \quad x = \sqrt{a}$$

(g) développement limité :

$$\sqrt{1+x} = 1 + \frac{1}{2}x - \frac{1}{2}\frac{x^2}{2} + \frac{3}{8}\frac{x^3}{6} + ...$$

2.
```
#include <iostream>
#include <iomanip>
#include <cmath>

namespace imac{

  double sqrt(const double value, const unsigned int nbIter){
   double u = 1.0;

   for(unsigned int i=0; i<nbIter; ++i)
       u = (u + (value/u)) * 0.5;

   return u;
  }

} // namespace

int main(int argc, char **argv)
{
  double a = 0;
  if(argc != 2){
   std::cerr << "usage: " << argv[0] << " nb" << std::endl;
   return 0;
  }

  a = atof(argv[1]);
  std::cout << "a       : " << a << std::endl;
  std::cout << std::setprecision(30) << "sqrt(a) : " << sqrt(a) << std::endl;
  std::cout << std::setprecision(30) << "sqrt(a) : " << imac::sqrt(a,10) << std::endl;

  return 0;
}
```

3. holala c'est encore chaud

- - - -